



UNIVERSIDAD CATÓLICA DEL MAULE

Facultad de Ciencias de la Ingeniería
Escuela de Ingeniería Civil Informática

EVALUACIÓN 1 - 20 %

ESTUDIO DE CASO - PARADIGMAS DE PROGRAMACIÓN

Ficha de Paradigma: Programación Funcional

Nombre de los Integrantes:

1. Joaquin Vasquez Gomez
2. Diego Solis Rojas
3. Javier Flores Espindola
4. Camilo Rizzo Cordova
5. Isidora Hormazabal Gallegos

Académico: Nicolás A. Reyes Reyes

Actividad Curricular: INF-223 Programación Avanzada

Fecha de Entrega: 31 de agosto de 2026

1. Definición del Paradigma y Conceptos del Lenguaje Núcleo

El paradigma de programación funcional se define en su núcleo mediante dos conceptos de programación fundamentales: los registros (*records*) y las clausuras de ámbito léxico (*lexically scoped closures*). Resulta importante destacar que no se basa en el manejo de estados mutables, sino en la evaluación de expresiones. Los registros actúan como la fundación de la programación simbólica, permitiendo agrupar referencias a datos. Por otro lado, la clausura léxica es el concepto que da vida al paradigma; esta permite empaquetar un conjunto de instrucciones de código junto con el entorno externo en el que fueron definidas. De este modo, el programa puede aislar ese “paquete de trabajo”, trasladarlo a cualquier otra parte del sistema y decidir ejecutarlo posteriormente, asegurando que se evalúe exactamente con el mismo contexto que tenía en su momento de creación (Van Roy, 2009, pp. 23-24).

2. Taxonomía y Concepto Diferencial

Observando la organización de los paradigmas, la programación funcional proviene de un paradigma vecino más sencillo: la “Programación funcional de primer orden” (*First-order functional programming*). El concepto clave y diferencial que se añade a este paradigma para evolucionar y llegar a la programación funcional completa es la **clausura** (*closure*). La adición de este elemento marca una frontera clara en la taxonomía de la evolución de los lenguajes (Van Roy, 2009, p. 13).

3. Expresividad y Razonamiento sobre el Programa

Al incorporar el concepto de la clausura léxica, la expresividad del lenguaje gana una enorme versatilidad. Específicamente, se obtiene la capacidad de separar la definición de un comportamiento de su ejecución. Esto permite crear estructuras de control personalizadas, instanciación, y facilita la programación basada en componentes (Van Roy, 2009, p. 24).

Sin embargo, debido a que el paradigma puro carece del concepto de “estado nombrado” (*named state*), se pierde cierta facilidad en la capacidad de razonar modularmente sobre entidades que intrínsecamente necesitan cambiar o recordar su pasado en el tiempo. Por lo tanto, podemos decir que, si un componente o función necesita llevar un conteo o actualizar una memoria interna, no puede simplemente mutar una variable oculta. En cambio, se ve forzado a modificar su interfaz para recibir y retornar el estado actualizado en cada llamada (Van Roy, 2009, pp. 26-28), lo que puede añadir una capa de complejidad cognitiva al razonar sobre flujos de información a largo plazo.

4. Evolución Histórica y Problema Técnico

La programación funcional surge de la necesidad de alcanzar un sistema Turing completo, esto es, un sistema computacional capaz de calcular cualquier función computable. El problema técnico radicaba en que, utilizando únicamente estructuras de datos puras (programación funcional de primer orden), no se alcanzaba esta completitud. Al integrar la clausura, el lenguaje se volvió equivalente al cálculo lambda (el cual sí es Turing completo).

Fue la necesidad técnica de tener funciones de primera clase, que pudiesen manipularse como cualquier otro dato, lo que motivó su aparición y consolidación (Van Roy, 2009, pp. 13-14).

5. Lenguajes Representativos y Justificación

Dos lenguajes clásicos y altamente representativos de este paradigma son **Scheme** y **Haskell** (Van Roy, 2009, p. 13). Scheme es un derivado de Lisp pionero en hacer de la clausura léxica una herramienta robusta y estándar. Por otro lado, Haskell representa la pureza del paradigma, incorporando ejecución perezosa (*lazy*) y garantizando inmutabilidad matemática estricta.

Para la resolución de nuestro caso de estudio utilizaremos **Kotlin** orientado a un estilo funcional. Elegimos Kotlin porque, aunque es multiparadigma, tiene un excelente manejo de funciones como ciudadanos de primera clase, ofreciendo lambdas y funciones de orden superior nativas que permiten aplicar la teoría de las clausuras sin fricciones, adaptándose a los estándares de desarrollo moderno.

6. Implementación del Caso de Estudio: Ventajas y Limitaciones

Al modelar el sistema de asignación de salas que procesa un flujo constante de solicitudes utilizando la programación funcional, emergen las siguientes particularidades:

Ventajas:

- **Cero condiciones de carrera:** Al no tener estado nombrado ni variables mutables (*race conditions*), es matemáticamente seguro procesar múltiples solicitudes de salas al mismo tiempo (conurrencia). La evaluación siempre será predecible.
- **Verificación determinista:** Debido a que las funciones son puras, si se pasa la misma solicitud de equipamiento y la misma lista de salas, siempre se obtendrá exactamente el mismo resultado, lo que facilita enormemente realizar pruebas de *software* (testing).
- **Declaratividad mediante funciones de orden superior:** Buscar la sala ideal filtrando por capacidad, franja horaria y equipamiento se logra de manera muy semántica y concisa usando clausuras (como en las funciones `filter` y `find`).

Limitaciones:

- **Arrastre manual del estado:** Dado que no podemos “sobrescribir” la disponibilidad de la sala asignada en una variable global, nos vemos obligados a pasar el catálogo de salas actualizado (el nuevo estado) como argumento en cada llamada recursiva o iteración del flujo de solicitudes.
- **Impacto en la memoria y rendimiento:** En la teoría pura, la inmutabilidad implica que, cada vez que una sala es asignada, se debe devolver una copia o una

nueva versión conceptual de la estructura de datos que refleje este cambio, lo cual demanda optimizaciones adicionales por parte del motor del lenguaje.

- **Barrera de pensamiento abstracto:** Implementar el manejo de un flujo continuo (*event-loop* o flujo de entrada) puramente mediante recursión y operaciones de orden superior suele requerir un sobreesfuerzo cognitivo inicial en comparación a programar un bucle condicional tradicional.

Anexo: Declaramos el uso de la Inteligencia Artificial

En cumplimiento con la normativa de uso responsable de IA de la asignatura, declaramos lo siguiente:

- **Herramienta utilizada:** Gemini (Google).
- **Link de Gemini:** [<https://share.gemini.google/aM0xhTtNSqMq>] (<https://share.gemini.google/aM0xhTtNSqMq>)
- **Link del repositorio GitHub:** [https://github.com/HulkGamerGt/Progra_avanzada] (https://github.com/HulkGamerGt/Progra_avanzada)
- **Propósito del uso:** Para organizar la estructura del documento, formatear en código LaTeX y revisar la fluidez de la redacción.
- **Aclaracion:** El contenido y las ideas principales son de autoría original. El texto final y las fuentes citadas fueron revisados y validados por el/los autores.

Referencias

- Van Roy, P. (2009). *Programming paradigms for dummies: what every programmer should know*. En New Computational Paradigms for Computer Music. IRCAM/Delatour. (Basado en el documento original proporcionado, pp. 9-47).